

기말고사 문제 1 — 홍익의 backward 고치기 (2-2-2, ReLU, Softmax+CE)

> ****기계학습 라이브러리 활용**** · 기말 Take-home · 배점 ****25점****

11주의 2-2-1(출력 1개, sigmoid, MSE) 네트워크를 ****2-2-2 + 은닉 ReLU + 출력 Softmax+CrossEntropy**** 로 바꿨다.

$$x \rightarrow \{W^{(1)}, b^{(1)}\} z^{(1)} \rightarrow \{\mathrm{ReLU}\} a^{(1)}$$
$$\rightarrow \{W^{(2)}, b^{(2)}\} z^{(2)} \rightarrow \{\mathrm{softmax}\} p, \quad L = -\sum_i y_i \log p_i$$

****상황****: ****홍익****이 이 네트워크의 **backward** 를 나름대로 계산해서 코드로 적었다.

그런데 ****원하는 gradient 값(정답 y)**** 이 나와야 하는데 ****엉뚱한 값(z)**** 이 나온다.

홍익이 틀린 줄은 **`# ← BUG`** 로 표시해 두었다. ****그 줄들을 고쳐 정답이 나오게 하라.****

| Part | 할 일 | 배점 |

|-----|-----|-----|

| 1 | `MLP2` forward 빈칸(ReLU-softmax-loss) 채우기 | 5 |

| 2 | 홍익의 backward 버그 3곳 수정 | 15 |

| 3 | 개념 관련 객관식 문제 | 5 |

****제약****: **numpy** 만. 코드를 처음부터 짜는 문제가 아니라 — 표시된 곳만 고칠 것!

 **학번 입력 (필수)**

```
```python
STUDENT_ID = "3085" # ← 본인 학번 마지막 4자리
assert STUDENT_ID != "0000", "학번을 마지막 4자리를 입력하세요!"
print("학번:", STUDENT_ID)
```
```

```text

학번: 3085

```

환경 준비

```
```python
import numpy as np
import matplotlib.pyplot as plt
```
```

Part 1. forward 빈칸 채우기 (5점)

은닉 활성화가 **ReLU** 로 바뀐 것에 주의. `\_relu`, `\_softmax`, `loss` 세 줄만 채워라.

- `\_relu(z)`: `max(0, z)` (numpy: `np.maximum`)
- `\_softmax(z)`: `z - max(z)` 후 `exp / 합`
- `loss(cache, y)`: CrossEntropy $-\sum y_i \log p_i$ (`+1e-12`)

```
```python
class MLP2:
 def __init__(self, random_seed=None):
 if random_seed is not None: np.random.seed(random_seed)
 self.W1=np.random.randn(2,2)*0.5; self.b1=np.zeros(2)
 self.W2=np.random.randn(2,2)*0.5; self.b2=np.zeros(2)

 def _relu(self, z):
 return np.maximum(0.0, z)

 def _softmax(self, z):
 z=z-np.max(z); e=np.exp(z); return e/np.sum(e)

 def forward(self, x):
 a1 = self._relu(self.W1 @ x + self.b1)
 return self._softmax(self.W2 @ a1 + self.b2)

 def forward_with_cache(self, x):
 z1=self.W1@x+self.b1; a1=self._relu(z1); z2=self.W2@a1+self.b2; p=self._softmax(z2)
 return {"x":x, "z1":z1, "a1":a1, "z2":z2, "p":p}

 def loss(self, cache, y):
 return -np.sum(y*np.log(cache["p"]+1e-12))
```

```
_m=MLP2(random_seed=0); _p=_m.forward(np.array([0.5, -0.3]))
```

```

print("forward 합 =", round(float(np.sum(_p)),6)) # output의 합이 1인지 확인
```

```text
forward 합 = 1.0
```

```

Part 2. 홍익의 backward 고치기 (15점)

아래 도구를 먼저 실행하라.

```

```python
=====
[수정 금지] 정답 forward + 도구. 그대로 실행하세요.
=====
def relu(z): return np.maximum(0.0, z)
def relu_prime(z): return (z > 0).astype(float) # ReLU 미분: z>0 → 1, 아니면 0

class MLP2Ref:
 """2-2-2, 은닉 ReLU, 출력 Softmax+CE. (정답 forward)"""
 def __init__(self, random_seed=None):
 if random_seed is not None: np.random.seed(random_seed)
 self.W1=np.random.randn(2,2)*0.5; self.b1=np.zeros(2)
 self.W2=np.random.randn(2,2)*0.5; self.b2=np.zeros(2)
 def _softmax(self,z): z=z-np.max(z); e=np.exp(z); return e/np.sum(e)
 def forward_with_cache(self,x):
 z1=self.W1@x+self.b1; a1=relu(z1); z2=self.W2@a1+self.b2; p=self._softmax(z2)
 return {"x":x,"z1":z1,"a1":a1,"z2":z2,"p":p}
 def forward(self,x): return self.forward_with_cache(x)["p"]
 def loss(self,cache,y): return -np.sum(y*np.log(cache["p"]+1e-12))

def gradient_check(mlp, backward_fn, x, y, eps=1e-5):
 ana=backward_fn(mlp, mlp.forward_with_cache(x), y)
 def L(): return mlp.loss(mlp.forward_with_cache(x), y)
 per={}
 for nm,par,g in [("W1",mlp.W1,ana["dW1"]),("b1",mlp.b1,ana["db1"]),
 ("W2",mlp.W2,ana["dW2"]),("b2",mlp.b2,ana["db2"])] :
 it=np.nditer(par,flags=["multi_index"],op_flags=["readwrite"]); mx=0.0
 while not it.finished:
 i=it.multi_index;o=par[i];par[i]=o+eps;lp=L();par[i]=o-eps;lm=L();par[i]=o
 num=(lp-lm)/(2*eps); mx=max(mx,abs(num-g[i])/max(abs(num),abs(g[i]),1e-12)); it.iternext()
 per[nm]=mx
 return per

----- 고정 테스트 케이스: "원하는 결과(정답)" -----
def make_case():
 m=MLP2Ref()
 m.W1=np.array([[0.1,0.2],[0.3,0.4]]); m.b1=np.zeros(2)
 m.W2=np.array([[0.5,0.6],[0.7,0.8]]); m.b2=np.zeros(2)
 x=np.array([1.0,1.0]); y=np.array([1.0,0.0])
 return m, x, y

```

```

EXPECTED = { # 정답 gradient (원하는 결과 y)
 "dw2": [[-0.165,-0.3849],[0.165,0.3849]],
 "db2": [-0.5498,0.5498],
 "dw1": [[0.11,0.11],[0.11,0.11]],
 "db1": [0.11,0.11],
}
def compare_to_expected(backward_fn):
 m,x,y=make_case(); g=backward_fn(m, m.forward_with_cache(x), y)
 print(f"{'param':5s} | {'홍익 결과(z)':28s} | 정답(y)")
 print("-"*70); allok=True
 for k in ["dw2","db2","dw1","db1"]:
 got=np.round(g[k],4); exp=np.array(EXPECTED[k]); ok=np.allclose(got,exp,atol=1e-3)
 allok&=ok
 print(f"{k:5s} | {str(got.tolist()):28s} | {exp.tolist()} {'✅' if ok else '❌'}")
 print("\n→ 모두 정답과 일치!" if allok else "\n→ 아직 다릅니다. 표시된 줄을 고치세요.")
 return allok
print("도구 준비 완료.")
```


``text  

    도구 준비 완료.  

    ```


```

## ## 홍익의 backward — 버그 3곳 (# ← BUG)

표시된 줄을 고쳐, `compare\_to\_expected(backward\_honggik)` 가 \*\*모두 ✅\*\* 가 되도록 하라.

- \*\*BUG1 (출력층)\*\*: Softmax+CE 의 출력 오차  $\partial L/\partial z^2$  는 \*깔끔\*하다.
- \*\*BUG2 (은닉층)\*\*: 출력이 2개다. 은닉 뉴런으로 들어오는 에러는 \*\*두 출력 경로의 합\*\*인데, 홍익은 한 경로만 더했다.
- \*\*BUG3 (ReLU)\*\*: 은닉 활성화가 ReLU 인데 홍익은 \*sigmoid\* 미분을 썼다.

```

```python
def backward_honggik(mlp, cache, y):
    x, z1, a1, p = cache["x"], cache["z1"], cache["a1"], cache["p"]

    # 1. 출력층 오차
    dz2 = p - y
    dw2 = np.outer(dz2, a1)
    db2 = dz2
    # ← BUG1 수정: Softmax+CE

    # 2. 은닉층으로 에러 역전파 (출력 0 경로 + 출력 1 경로 = 합!)
    dL_da1_0 = mlp.W2[0,0]*dz2[0] + mlp.W2[1,0]*dz2[1] # ← BUG2 수정
    dL_da1_1 = mlp.W2[0,1]*dz2[0] + mlp.W2[1,1]*dz2[1] # ← BUG2 수정
    dL_da1 = np.array([dL_da1_0, dL_da1_1])

    # 3. 활성화 미분 통과 (ReLU!)

```

```
delta1 = dL_da1 * relu_prime(z1) # ← BUG3 수정
```

```
# 4. 은닉층 파라미터
```

```
dw1 = np.outer(delta1, x)
```

```
db1 = delta1
```

```
return {"dw1": dw1, "db1": db1, "dw2": dw2, "db2": db2}
```

```
compare_to_expected(backward_honggik)
```
```

```
```text
```

```
param | 홍익 결과(z) | 정답(y)
```

```
-----
dw2 | [[-0.165, -0.3849], [0.165, 0.3849]] | [[-0.165, -0.3849], [0.165, 0.3849]] ✓
db2 | [-0.5498, 0.5498] | [-0.5498, 0.5498] ✓
dw1 | [[0.11, 0.11], [0.11, 0.11]] | [[0.11, 0.11], [0.11, 0.11]] ✓
db1 | [0.11, 0.11] | [0.11, 0.11] ✓
```

```
→ 모두 정답과 일치!
```

```
```
```

```
```text
```

```
True
```

```
```
```

### ### (선택, 채점 외) 랜덤 입력으로도 검증

`compare\_to\_expected` 는 고정 케이스만 본다. 아래는 임의 입력에서 수치미분과 비교한다(모두 < 1e-7 이면 완벽).

```
```python
```

```
np.random.seed(7); _mr=MLP2Ref(random_seed=7); _xr=np.random.randn(2)
```

```
_per=gradient_check(_mr, backward_honggik, _xr, np.array([1.0,0.0]))
```

```
print({k: f"{v:.1e}" for k,v in _per.items()})
```

```
```
```

```
```text
```

```
{'W1': '5.7e-11', 'b1': '1.6e-11', 'W2': '6.8e-11', 'b2': '1.4e-11'}
```

```
```
```

```

```

### # Part 3. 개념 객관식 (5점)

Q1~5번 객관식 문제의 답을 다음 셀 `concept\_answers` 디렉터리의 각 문제 키 값으로 작성하시오.  
(None` 부분을 `a`~`d` 로 수정)

**\*\*Q1.\*\* Softmax+CE 의 출력층 오차  $\partial L/\partial z^2$  는? <br>**

&nbsp;(a)  $p-y$  &nbsp;(b)  $(p-y)p(1-p)$  &nbsp;(c)  $y-p$  &nbsp;(d)  $(p-y)a^1$

**\*\*Q2.\*\* 출력이 \*\*2개\*\*일 때, 은닉 뉴런  $a^1_j$  로 들어오는 에러  $\partial L/\partial a^1_j$  는? <br>**

&nbsp;(a) 출력 0 경로만 &nbsp;(b) 출력 1 경로만 &nbsp;(c) \*\*두 출력 경로의 합\*\* &nbsp;(d) 두 경로의 곱

**\*\*Q3.\*\* 은닉 활성화가 ReLU 일 때  $\delta^1 = \partial L/\partial a^1 \cdot ?$  (단,  $z^1$  은 ReLU 입력) <br>**

&nbsp;(a)  $a^1(1-a^1)$  &nbsp;(b)  $(z^1>0)$  &nbsp;(c) 1 &nbsp;(d)  $z^1$

**\*\*Q4.\*\* 홍익이 BUG2(출력 1 경로 누락)만 가지고 있을 때, 값이 틀려지는 gradient 는? <br>**

&nbsp;(a)  $dW2, db2$  &nbsp;(b)  $dW1, db1$  &nbsp;(c) 전부 &nbsp;(d) 없음

**\*\*Q5.\*\* 출력층 오차  $dz^2(=BUG1)$  가 틀리면 영향 범위는? <br>**

&nbsp;(a)  $dW2, db2$  만 &nbsp;(b)  $dW1, db1$  만 &nbsp;(c) 네 파라미터 전부 &nbsp;(d) 없음

```
```python
concept_answers = {"q1":"a", "q2":"c", "q3":"b", "q4":"b", "q5":"c"} # 'a'~'d'
```
```

---

#  제출 전 체크리스트

- [ ] 학번 입력 / Part 1 forward 합 = 1
- [ ] `compare_to_expected(backward_honggik)` 가 \*\*모두  \*\*
- [ ] `concept_answers` q1~q5 채움
- [ ] 파일명 `[학번]_problem1.ipynb`